



Instructor: Date:

Learner: Regional Branch:

Program: Center:

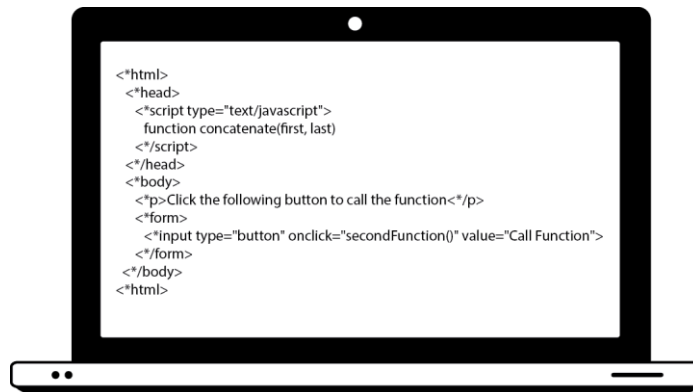
CONTENT

- Let's Start!
- Let's Explore!
- Let's Practice!
- Extension Activity
- Target Words
- Learning Points



Let's Start!

Functions in JavaScript



Learning Outcome:

Understand information about web applications, their principles and basic concepts in a range of work contexts. Describe applications for an operating system for personal computers in a range of work contexts.

Can Do Statements:

- I can say what a function in Javascript is.
- I can discuss how to invoke a function in Javascript.
- I can use function parameters in JavaScript correctly.

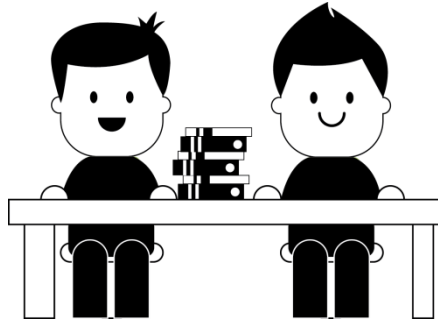
Ask your partner:

- What do you think the difference between Java and JavaScript is?
- Why do you think it is important to work with JavaScript when working with dynamic websites?



Let's Explore!

Mark is helping Kelly to understand some JavaScript functions. Can you identify what they are?



Instructions: Read the text below.

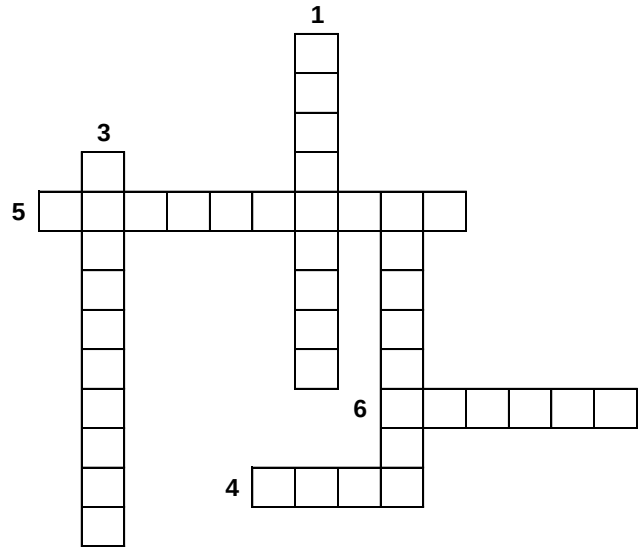
- Kelly:** Hi, Mark. I know you are tech-savvy, so I was wondering if you could help with something here.
- Mark:** Sure, tell me, what am I good for?
- Kelly:** To pass the exam on JavaScript, I need to know what a function is; could you help me to understand that?
- Mark:** Rest easy, Kelly. I'll explain to you what it is. Listen, programming used to be harder, but it's now much simpler than it used to be thanks to function parameters. Functions are a group of reusable code; so, you only have to write it once and you can use it as many times as you feel it necessary. You can divide a big program into small and manageable functions. To show an example of this, let's use the function "alert."
- Kelly:** I saw that one in a previous class. I think I have that one clear.
- Mark:** I hope so. When you use such a function, you only need to write it once in your code and call it back with a return statement, believe me, it will help you save tons of time.
- Kelly:** Ok... Let me see if I got it; we can write basic codes into groups of modular codes called functions, and we can use them over and over again by calling them, writing their names, right?
- Mark:** Exactly! Now, time to see how to define a function. We assign the function a name and put a list of parameters and statement blocks surrounded by curly braces.
- Kelly:** The statement block is something I studied in last class, so I remember that one.
- Mark:** Perfect! Then I won't explain it. When you create a function, you can use it to concatenate different pieces of data, codes or even other functions and parameters; for instance, a mathematical operation such as "3 * 3" or any other operation you need to perform with the information you've gathered from the database by means of other scripts.
- Kelly:** Wait. You said that we put a list of parameters separated by commas. Aren't parameters and functions different things?
- Mark:** They are, but we put them inside the functions and manipulate them according to our needs, which helps us save time, too. Finally, to use functions, you must also know what a return statement is; basically, it returns the value from a function to its original value, before it is applied to the calling program.
- Kelly:** Ok, I'll be sure to remember that.

**Let's Practice!**

Let's see what JavaScript is.

Instructions: Read the clues. Complete the crossword.

1. You'll need to know what f _ n _ _ I _ _ _ are if you want to code well with JavaScript.
2. Functions are a group of r _ _ s _ _ l _ code that you can call any time you need.
3. As you can see, we divided this program into many m _ _ _ g _ _ b _ e functions to make it easy to understand.
4. You only need to write a function o _ _ e and call it as many times you need it during the coding process.
5. We include the p _ _ a _ _ _ e _ s inside the functions and manipulate them according to our needs.
6. You surrounded all the statement blocks with curly b _ _ c _ _.

**Let's Practice!**

Characteristics of invoking a function.

Instructions: Organize the mystery sentence.

1. To / a / the / need / write / to / once. / only / function / function, / you / call

2. functions / the / groups. / of / organize / into / codes / inside / To / basic / we / put / code, / strings

3. a / assign / make / of / to / call / a / needed. / function, / we / To / it / to / use / name / need / and / when / it



Extension Activity

It is time for you to show how much you've learnt so far about JavaScript.

Instructions: Complete the following steps. Fill in the Self-Assessment.

- Find on the web different examples of functions in JavaScript.
- Select three functions you would like to write.
- Use parameters and statement blocks.
- Show and explain to your classmates how you did it.

Self-Assessment

Yes No

- I find different examples of functions.
- I write functions correctly
- I show my examples to my partners and explain how I did it.

☐	☐
☐	☐
☐	☐



Target Words

- **rest easy:** (expression): not to worry about something
e.g. Rest easy, Kelly. I'll explain to you what it is. Listen, programming used to be harder, but it's now much simpler than it used to be thanks to function parameters.
- **reusable** (adjective): that can be used more than once
e.g. Functions are a group of reusable code; so, you only have to write it once and you can use it as many times as you feel it necessary.
- **save** (verb): to reduce the amount a resource is used, to accumulate and prevent from using
e.g. When you use such a function, you only need to write it once in your code and call it back with a return statement, believe me, it will help you save tons of time.
- **for instance** (adverb): another way to say "for example"
e.g. You can use it to concatenate different pieces of data, codes or even other functions; for instance, a mathematical operation such as "3 * 3"
- **according to** (preposition): something that depends on the following statement
e.g. They are, but we put them inside the functions and manipulate them according to our needs, which helps us save time, too.



Learning Points

Review of Simple Tenses

We use **simple past**, **simple present** and **simple future** with different purposes.

Past Simple

We use the past simple time tense to express those actions that happened and finished in the past.

Structure

Subject + verb in past + complement

Examples:

*I **finished** programming the company's website this morning at around 3 am.*

*I **wrote** this app's code; It **took** me a month, but it is sleek.*

Present Simple

We use the present simple tense to talk about routines and general facts.

Structure:

Subject + verb + complement

Example:

*Our company **specializes** in designing web based applications for companies like yours.*

*I **work** full time as a program designer.*

Future Simple (Will)

We use the future simple tense with **will** to talk about actions that are happening in the future or decisions we make at the moment of speaking.

Structure

Subject + will + verb (base form) + Complement

*I **will finish** programming this app tomorrow, I'm leaving now.*

A: Who wants to build an app for XXX ltd?

*B: I'll **do** it if you give me the chance.*

Examples

- *You only need to write it once in you code and call it back with a return statement, believe me, it will help you save tons of time.*
- *We can write basic codes into groups of modular codes called functions, and we can use them over and over again by calling them, writing their names, right?*
- *I was wondering if you could help with something here.*





Learning Points

To + Verb to Start a Sentence

You can use the infinitive of purpose to start a sentence showing the purpose of the following action first.

Bear in mind that the infinitive of purpose is a dependent clause, that is, a clause or idea that is not complete and depends on a full idea to make sense.

You usually differentiate them because a full idea starts with a subject and follows the next pattern: **Subject + Verb + Complement**. However, a dependent clause can start with a preposition or adverb; therefore, you never separate them with a comma, as they do not have full meaning.

The infinitive of purpose usually goes after the main idea it gives the reason of existence.

Example:

*You must use JavaScript **to make** the website dynamic.*

*We've implemented some scripts **to make** the page more dynamic for the users.*

Notice that both ideas are together without a comma. However, if you start the sentence by using the infinitive of purpose, you must separate them with a comma.

Examples:

***To make** the website dynamic, you must use JavaScript.*

***To make** the page more dynamic for the users, we've implemented some scripts.*

Examples

- *To pass the exam on JavaScript, I need to know what a function is.*
- *Finally, to use functions, you must also know what a return statement is.*



Learning Points

The Return Statement in JavaScript.

The return statement helps you call a previous function in a JavaScript line of code.

As you probably know, JavaScript is used to add interaction to a webpage by means of codes; therefore, each code concatenates a specific action within a page. An example being the **return** statement.

By implementing the **return** statement, you guarantee the webpage to call or return the value of a previous function named in your code, such function can either be a mathematical function or not.

For example:

In the following example we will create a button that will concatenate a mathematical result coming from a series of values that you could have gathered from a database or a different box within the page.

```
<*html>
  <*head>
    <*script type="text/javascript">
      function concatenate(first, last)
      {
        var full;
        full = first * last;
        return full;
      }

      function secondFunction()
      {
        var result;
        result = concatenate('3', '3');
        document.write (result );
      }
    <*/script>
  <*/head>

  <*body>
    <*p>Click the following button to call the function<*/p>

    <*form>
      <*input type="button" onclick="secondFunction()" value="Call Function">
    <*/form>
  <*/body>
<*/html>
```

If you copied the previous line of code into a text editor, the result would be a button whose internal code would make the operation $3 * 3 = 9$.